UNIVERSIDAD POLITÉCNICA DE CHIAPAS
INGENIERÍA MECATRÓNICA

SISTEMAS DE VISIÓN ARTIFICIAL
Tarea 5. Reporte de Caso de Estudio
Corte 2

Docente:
Edgar gonzalez Santiago

Alumno:
233419 - Rojas Rodríguez Roberto

Grupo: 8B
Fecha: 03/03/2026
Suchiapa, Chiapas

ÍNDICE

INTRODUCCIÓN **3**
1. Análisis y Arquitectura del Sistema **4**
1.1 Lógica de Negocio 4
1.2 Diagrama de flujo 4
2. Especificaciones técnicas **6**
2.1 Requerimientos de Software y Librerías 6
2.2 Estructuras de Datos 6
2.2 Justificación de Funciones de OpenCV 6
2.3 Estándares de Código 7
2.4 Implementación y Pruebas 9
3. Conclusiones y Optimización **11**
3.1 Análisis de la Implementación 11
3.2 Posibles Mejoras 11

INTRODUCCIÓN

Este reporte documenta el desarrollo y la implementación de un sistema de visión artificial diseñado para la inspección automática de calidad de una llave de ajuste. El propósito principal es verificar que cada pieza cumpla con los estándares de manufactura antes de continuar en la línea de producción.

Problema

El desafío técnico consiste en sustituir la inspección visual manual, frecuentemente subjetiva y propensa a errores, por un método objetivo y repetible. Se requiere validar tres aspectos críticos en tiempo real:

- Integridad física: Confirmar que la silueta de la llave esté completa.
- Ausencia de defectos: Detectar irregularidades como rebabas, grietas o roturas parciales que comprometan la funcionalidad .
- Control dimensional: Asegurar que el tamaño de la pieza sea consistente, utilizando el área de la silueta como indicador básico (el cual debe ser $\geq 95\%$ del valor nominal de 25,000 píxeles).

La solución propuesta utiliza la librería OpenCV en Python para procesar imágenes, segmentar la pieza del fondo y realizar un análisis geométrico que permita clasificar cada unidad como APTA o NO APTA. Este enfoque no solo asegura el cumplimiento de normas de calidad como ISO 9001 / IATF 16949, sino que también proporciona evidencia visual para auditorías y mejora continua del proceso.

1. Análisis y Arquitectura del Sistema

En este apartado se detalla la planeación lógica y estructural del programa de inspección de calidad antes de su implementación en código.

1.1 Lógica de Negocio

El programa está diseñado bajo un modelo de decisión binaria ("Pasa / No Pasa") basado en criterios industriales de inspección visual. Las reglas que rige el sistema son:

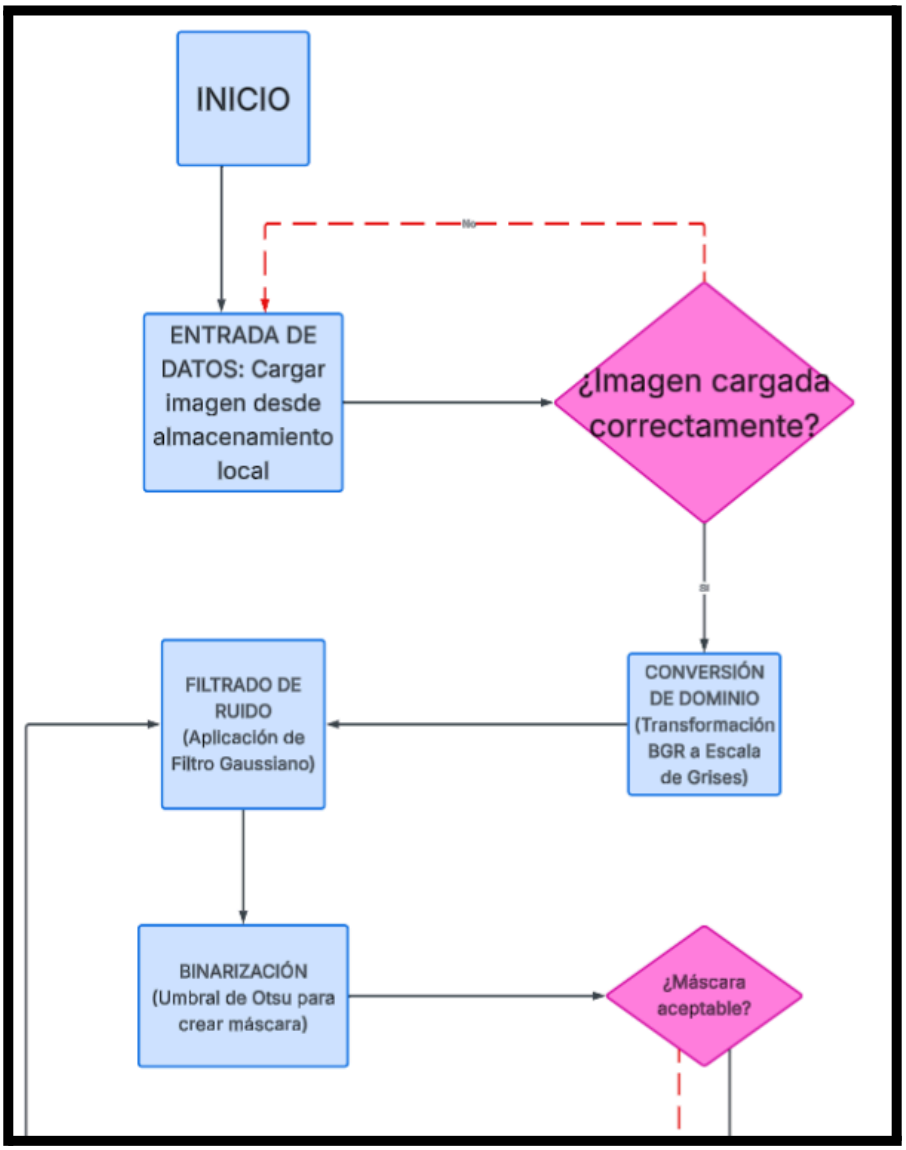
1. Regla de Segmentación: La llave debe ser claramente diferenciable del fondo. Se asume un contraste suficiente para que el algoritmo de Otsu pueda separar el objeto de interés.

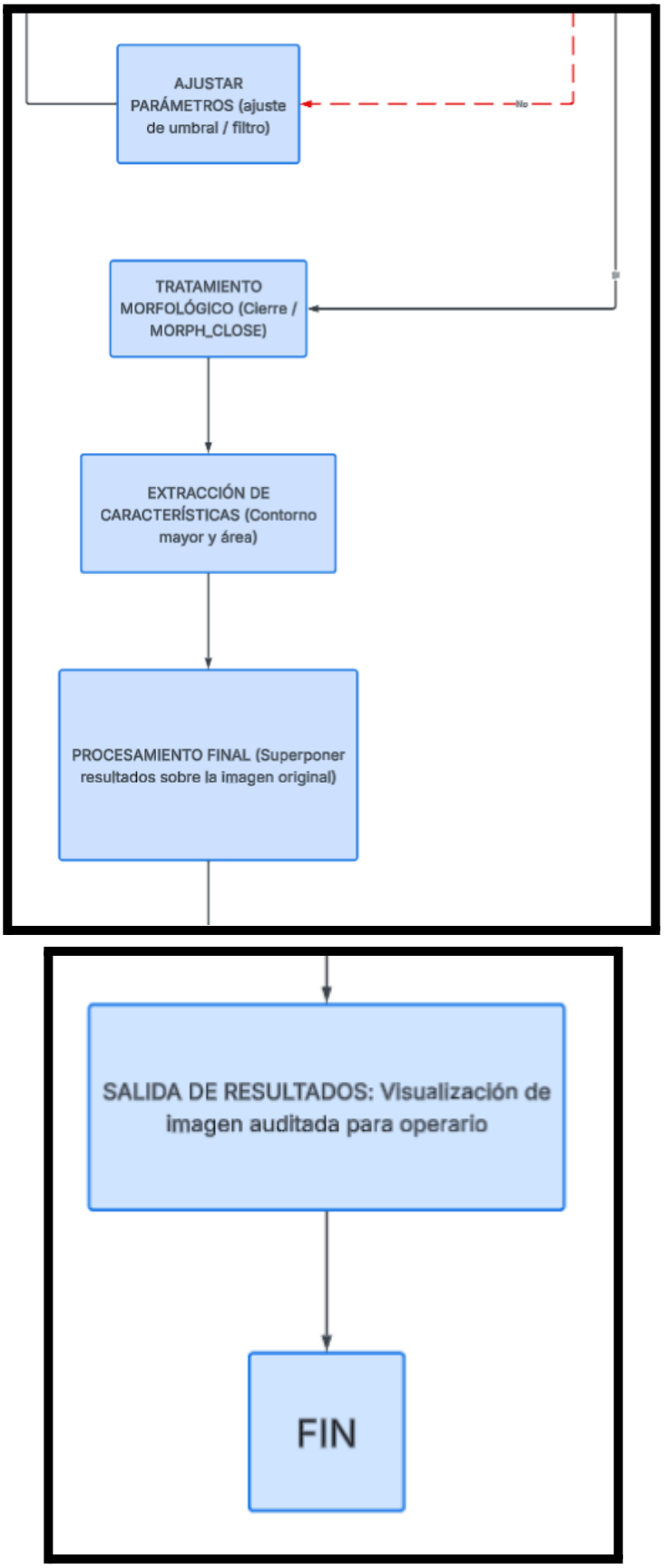
2. Criterio de Aceptación Dimensional: Se define un Área Nominal de 25,000 píxeles. El programa calcula el área real de la pieza detectada y aplica la condición:

    - Área 95% del valor nominal: Pieza APTA.
    - Área 95% del valor nominal: Pieza NO APTA.

3. Regla de Integridad: Se utiliza la función de contornos externos para asegurar que se analiza la pieza completa y no fragmentos pequeños de ruido.

1.2 Diagrama de flujo




AJUSTAR PARÁMETROS (ajuste de umbral / filtro)
No
TRATAMIENTO MORFOLÓGICO (Cierre / MORPH_CLOSE)
EXTRACCIÓN DE CARACTERÍSTICAS (Contorno mayor y área)
PROCESAMIENTO FINAL (Superponer resultados sobre la imagen original)
SALIDA DE RESULTADOS: Visualización de imagen auditada para operario
FIN

2. Especificaciones técnicas

Para que el sistema de visión artificial funcione de manera óptima, se ha definido un "ecosistema" tecnológico basado en librerías estándar de la industria y estructuras de datos eficientes.

2.1 Requerimientos de Software y Librerías

El proyecto se ejecuta sobre Python 3.11.9 y utiliza las siguientes dependencias externas:

- OpenCV (cv2): Es la librería principal encargada de la carga de imágenes , conversión a escala de grises , suavizado Gaussiano y la segmentación por el método de Otsu .

- NumPy: Utilizada para el manejo de la imagen como una matriz numérica y para la creación del "kernel" (matriz de 5×5) necesario en las operaciones de limpieza morfológica.

2.2 Estructuras de Datos

La elección de las estructuras de datos dentro del código está justificada por la necesidad de procesar información espacial de forma rápida:

- Matrices (Arrays de NumPy): La imagen se trata como una matriz de píxeles. Esto permite aplicar operaciones matemáticas (como la umbralización) a millones de puntos de datos simultáneamente.

- Listas de Contornos: Los resultados de la función findContours se almacenan en una estructura de lista. Cada elemento de la lista representa un objeto detectado en la imagen.

- Vectores de Puntos: El contorno específico de la llave se guarda como un conjunto de coordenadas (x, y) que definen su perímetro, permitiendo el cálculo exacto de su área geométrica mediante la función

2.2 Justificación de Funciones de OpenCV

Se seleccionaron funciones específicas para garantizar la objetividad del estándar de calidad:

- Otsu Thresholding: Se eligió para eliminar la subjetividad humana al elegir un umbral de luz; el algoritmo calcula automáticamente el punto óptimo para separar la llave del fondo.

- Morphological Close: Se utilizó para "rellenar" pequeñas imperfecciones o sombras dentro de la llave, asegurando que el cálculo del área no se vea afectado por ruido visual .

2.3 Estándares de Código

Para garantizar que el software sea mantenible, legible y profesional, se han aplicado los siguientes estándares de programación basados en las mejores prácticas de la industria y la guía PEP 8.

Tabla de Cumplimiento de Estándares

Elemento	Requisito	Aplicación en el Proyecto
Estilo	Adherencia estricta a PEP 8	Uso de sangrías de 4 espacios y separación clara entre bloques lógicos.
Documentación	Uso de Docstrings (""...""")	Se han incluido descripciones en el encabezado de las funciones principales.
Naming	**snake_case**	Las variables y funciones siguen el formato area_detectada o binaria_limpia.
Tipado	**Type Hints**	Uso de anotaciones como np.ndarray y float para definir tipos de datos.

```
import cv2
import numpy as np
from typing import Tuple, List, Optional

def procesar_inspeccion(ruta_imagen: str, area_nominal: int = 25000) -> None:
    """
    Realiza la inspección de calidad de una llave de ajuste.

    Args:
        ruta_imagen (str): Ruta del archivo de imagen.
        area_nominal (int): Área de referencia en píxeles.
    """
    # 1. Manejo de Errores: Carga de imagen
    imagen: Optional[np.ndarray] = cv2.imread(ruta_imagen)

    if imagen is None:
        print(f"Error: No se pudo cargar la imagen en {ruta_imagen}")
        return

    # 2. Preprocesamiento
    gris: np.ndarray = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
    suave: np.ndarray = cv2.GaussianBlur(gris, (5, 5), 0)

    # 3. Segmentación (Otsu)
    _, binaria = cv2.threshold(
        suave, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
    )

    # 4. Limpieza Morfológica
    kernel: np.ndarray = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    binaria_limpia: np.ndarray = cv2.morphologyEx(binaria, cv2.MORPH_CLOSE, kernel)

    # 5. Análisis de Contornos
    contornos, _ = cv2.findContours(
        binaria_limpia, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
```

```
    if not contornos:
        print("No se detectaron contornos en la imagen.")
        return

    # Selección del contorno mayor
    contorno_llave = max(contornos, key=cv2.contourArea)
    area_detectada: float = cv2.contourArea(contorno_llave)

    # 6. Evaluación de Calidad
    porcentaje_area: float = (area_detectada / area_nominal) * 100
    es_apta: bool = porcentaje_area >= 95

    # Configuración de salida
    estado: str = "APTA" if es_apta else "NO APTA"
    color: Tuple[int, int, int] = (0, 255, 0) if es_apta else (0, 0, 255)

    # 7. Visualización de Resultados
    cv2.drawContours(imagen, [contorno_llave], -1, color, 2)
    cv2.putText(imagen, f"Estado: {estado}", (30, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    cv2.putText(imagen, f"Area: {int(porcentaje_area)}%", (30, 80),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

    cv2.imshow("Resultado Inspeccion", imagen)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

if __name__ == "__main__":
    procesar_inspeccion("llave_ajuste.jpg")
```

2.4 Implementación y Pruebas

Evidencia de Ejecución

El sistema fue probado utilizando una imagen de una llave de ajuste de 250mm para validar la efectividad de las etapas de procesamiento y la precisión del diagnóstico final.

Segmentación Binaria

En esta etapa, el algoritmo de Otsu y la limpieza morfológica logran aislar la pieza del fondo . Como se observa en la imagen de control, se obtiene una máscara blanca que representa la superficie detectada, eliminando sombras y reflejos que podrían falsear el cálculo del área.

Diagnóstico Final

La imagen de salida muestra la superposición del contorno detectado en color verde, indicando que la pieza cumple con los criterios dimensionales. El sistema despliega en la esquina superior izquierda los indicadores clave:

Estado: APTA

Área Detectada: En este caso, el valor calculado superó ampliamente el umbral mínimo del 95% respecto al valor nominal configurado.

Caso de Prueba	Entrada	Descripción	Resultado Obtenido
Validación de Área	Llave 250mm	Comparación contra área nominal de 25,000 px.	APTA (Área satisfactoria).
Detección de Forma	Contorno Externo	Extracción del perímetro mediante RETR_EXTERNAL.	Éxito: Contorno continuo y sin huecos .

3. Conclusiones y Optimización

3.1 Análisis de la Implementación

La evidencia visual confirma que el uso de la umbralización de Otsu es efectivo para este tipo de herramientas metálicas sobre fondos claros . La detección del contorno es precisa, logrando capturar detalles como el relieve de la marca "250mm" sin comprometer la integridad de la silueta principal.

3.2 Posibles Mejoras

- Calibración de Cámara: Para entornos industriales, se recomienda convertir el área de píxeles a milímetros cuadrados reales para una mayor precisión dimensional.

- Control de Iluminación: Aunque la segmentación fue exitosa, la presencia de reflejos en el metal sugiere el uso de luz difusa para evitar que el algoritmo interprete brillos intensos como huecos en la pieza